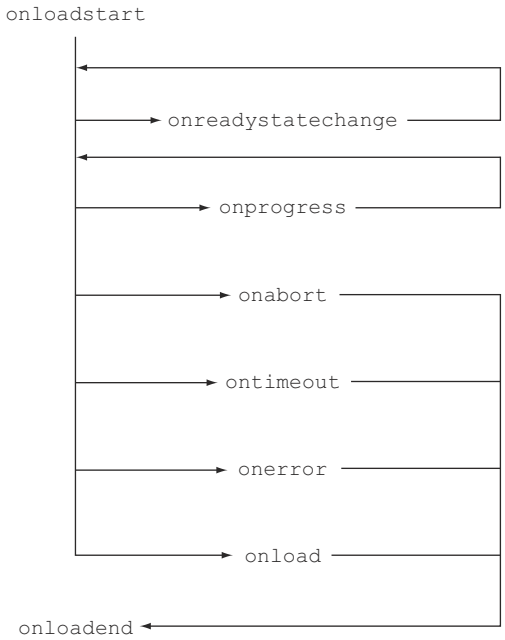


Figure 2.5 Which event handlers fire during the course of an HTTP request

The user can assign a function to each of these event handers to process the particular event. If you need to always run some code at the end of the request (regardless of whether or not there was an error), you'd assign a method to the `onloadend` event handler. The code isn't required to handle each of those events; if an event handler is unassigned, nothing happens when that event is fired. Let's take a closer look at some of the more important event handlers.

HANDLING A SUCCESSFUL RESPONSE WITH THE ONLOAD EVENT HANDLER

The `onload` event handler fires when the request is successful. In the Flickr example, the `onload` handler is responsible for displaying the photos on the page, as shown in the following listing.

Listing 2.4 Assigning a function to the `XMLHttpRequest`'s `onload` event handler

```
<!DOCTYPE html>
<html>
<body onload="loadPhotos();">
<div id="photos">Loading photos...</div>
<script>
function loadPhotos() {
  var api_key = '<YOUR API KEY HERE>';
  var method = 'GET';
  var url = 'https://api.flickr.com/services/rest/?' +
      'method=flickr.people.getPublicPhotos&' +
      'user_id=32951986%40N05&' +
      'extras=url_q&format=json&nojsoncallback=1&' +
      'api_key=' + api_key;
```